

Reinforcement Learning

100 Detailed Questions: MAB, MC, TD, DQN, Policy Methods

GARP RAI Exam Preparation

Which of the following statements most accurately describes the fundamental difference between reinforcement learning and supervised learning?

- ❶ Reinforcement learning uses labeled data to make predictions, while supervised learning learns through trial and error.
- ❷ Reinforcement learning learns what to do by mapping situations to actions through trial and error to maximize cumulative reward, while supervised learning learns to map inputs to outputs from labeled examples.
- ❸ Reinforcement learning is a subset of unsupervised learning, while supervised learning is a separate paradigm.
- ❹ Reinforcement learning uses no feedback, while supervised learning uses reward signals.

Answer: B Explanation: Option B correctly captures the essence: RL learns optimal actions through interaction (trial and error) to maximize cumulative reward, while supervised learning learns from labeled datasets. Option A reverses the definitions. Option C is incorrect (RL is a distinct paradigm). Option D is incorrect (RL uses rewards as feedback; supervised uses labels).

Exam Trap: RL is NOT supervised learning - it learns from experience, not from labeled data. The output of RL is a **policy** (strategy), not a prediction.

What is the "output" of a reinforcement learning model?

- ① A numerical prediction for a target variable.
- ② A classification label for a new observation.
- ③ A recommended action or a complete decision-making strategy (policy).
- ④ A set of clusters in the data.

Answer: C Explanation: Unlike supervised (prediction/classification) or unsupervised (clusters), RL outputs a policy - a strategy for decision-making. The agent learns to map states to actions. Option A describes regression, B describes classification, D describes clustering.

Exam Trap: Remember: Supervised → Predictions/Classifications; Unsupervised → Patterns/Clusters; Reinforcement → Actions/Policies!

Which of the following is NOT a typical application of reinforcement learning?

- ① Training robots to perform complex manipulation tasks.
- ② Developing policies for self-driving cars.
- ③ Predicting housing prices from historical data.
- ④ Dynamic pricing of airline seats based on supply and demand.

Answer: C Explanation: Predicting housing prices is a supervised learning (regression) task. Options A, B, and D are RL applications involving sequential decision-making and learning from interaction. RL is used in robotics, autonomous vehicles, robotics, autonomous systems, logistics, pricing, trading, and chatbots.

Exam Trap: RL is NOT for static prediction tasks - it's for [sequential decision-making](#) where actions affect future states and rewards!

In reinforcement learning, what is the "agent"?

- ❶ The environment in which decisions are made.
- ❷ The learner or decision-maker that takes actions to maximize reward.
- ❸ The reward signal received after taking an action.
- ❹ The set of all possible states in the environment.

Answer: B Explanation: The agent is the learner/decision-maker. Option A is the environment. Option C is the reward. Option D is the state space. The agent interacts with the environment by taking actions and receiving rewards, learning to improve its policy over time.

Exam Trap: The **agent** is the learner; the **environment** is what the agent interacts with. Don't confuse them!

In reinforcement learning, what is the "environment"?

- ❶ The algorithm used to update Q-values.
- ❷ The world in which the agent operates and from which it receives rewards.
- ❸ The policy that maps states to actions.
- ❹ The total expected future reward.

Answer: B Explanation: The environment is the external system the agent interacts with. It provides states and rewards in response to the agent's actions. Option A describes the learning algorithm. Option C is the policy. Option D is the value function.

Exam Trap: The **environment** is external to the agent - it's what the agent interacts with, not the agent itself!

In the Multi-Armed Bandit problem, what is an "action"?

- ① The payout received from playing a slot machine.
- ② The decision of which slot machine to play in a given round.
- ③ The total winnings after all rounds.
- ④ The probability of winning on a slot machine.

Answer: B Explanation: In MAB, an action is choosing which slot machine to play. Option A is the reward. Option C is the cumulative return. Option D is the reward probability (which is unknown to the agent). The set of actions includes all slot machines available.

Exam Trap: Actions are **choices** the agent makes, not the outcomes (rewards) of those choices!

In the Multi-Armed Bandit problem, what is the "state" of the environment?

- ❶ The current payout from a slot machine.
- ❷ The number of tokens the gambler has left.
- ❸ A single, constant state because the machines' properties do not change.
- ❹ The probability distribution of rewards for each machine.

Answer: C Explanation: In the simple MAB problem, since the slot machines' properties don't change and the agent's actions don't affect them, the environment is considered to be in a single, constant state. Option A is the reward. Option B is not part of the basic MAB. Option D is the reward distribution (unknown to the agent).

Exam Trap: In MAB, there is **only ONE state** - unlike more complex RL problems where states change based on actions!

What is a "reward" in reinforcement learning?

- ❶ The total cumulative return over all time steps.
- ❷ The feedback the agent receives from the environment after taking an action, which can be positive or negative.
- ❸ The policy that determines which action to take.
- ❹ The set of all possible states in the environment.

Answer: B Explanation: Reward is immediate feedback (positive or negative) received after an action. Option A is the cumulative return. Option C is the policy. Option D is the state space. Rewards are the signals that guide the agent's learning.

Exam Trap: **Reward** is immediate feedback; **Return** is the cumulative discounted sum of future rewards. Don't confuse them!

What is a "policy" in reinforcement learning?

- ❶ The algorithm used to update Q-values.
- ❷ The total expected future reward from a state.
- ❸ The agent's strategy or rule for choosing actions given the current state.
- ❹ The probability of transitioning from one state to another.

Answer: C Explanation: A policy is the strategy that maps states to actions. Option A describes the learning algorithm. Option B is the value function. Option D is the transition probability. The goal of RL is to find the optimal policy that maximizes cumulative reward.

Exam Trap: The policy is the **strategy** - the agent's "rule book" for what to do in each state!

What does the state-value function $V(s)$ represent?

- ① The immediate reward received from taking an action in state s .
- ② The total expected future reward an agent can receive starting from state s and following a specific policy.
- ③ The probability of being in state s .
- ④ The action that maximizes reward in state s .

Answer: B Explanation: $V(s)$ estimates the "goodness" of being in state s - the total expected future reward from that state under a policy. Option A is the immediate reward. Option C is the state distribution. Option D is the optimal action. $V(s) = E[G_t - S_t = s]$ where G_t is the cumulative future reward.

Exam Trap: $V(s)$ is **future** rewards from state s , NOT immediate reward!

What does the action-value function $Q(s,a)$ represent?

- ❶ The probability of taking action a in state s .
- ❷ The total expected future reward an agent can receive by taking action a from state s and following the policy thereafter.
- ❸ The immediate reward from taking action a in state s .
- ❹ The number of times action a has been taken in state s .

Answer: B Explanation: $Q(s,a)$ estimates how good it is to take action a in state s . It represents the expected cumulative future reward. Option A is the policy probability. Option C is the immediate reward. Option D is the visit count. $Q(s,a) = E[G_t - S_t = s, A_t = a]$.

Exam Trap: $Q(s,a)$ is **action-specific**; $V(s)$ is **state-specific**. If you know $Q(s,a)$ for all actions, the optimal policy is to choose the action with the highest Q -value!

In the discounted sum of future rewards $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$, what does γ represent?

- 1 The learning rate.
- 2 The discount factor that determines how much future rewards are valued relative to immediate rewards.
- 3 The exploration rate.
- 4 The probability of transitioning to a new state.

Answer: B Explanation: γ (gamma) is the discount factor, $0 \leq \gamma \leq 1$. It determines the value of future rewards relative to immediate rewards. A γ close to 0 values immediate rewards highly; γ close to 1 values future rewards similarly to immediate ones. Option A is α (learning rate). Option C is ϵ (exploration rate). Option D is transition probability.

Exam Trap: γ = discount factor; α = learning rate; ϵ = exploration rate. Don't confuse these three hyperparameters!

What is the "greedy strategy" in the Multi-Armed Bandit problem?

- 1 Always choosing a random slot machine to explore.
- 2 Always choosing the slot machine with the best record of payouts so far (exploitation only).
- 3 Alternating between exploration and exploitation.
- 4 Choosing the slot machine that has been played the least.

Answer: B Explanation: The greedy strategy always exploits - it picks the action with the highest estimated Q-value so far.

Option A is the random strategy. Option C is ϵ -greedy. Option D is not a standard strategy. The greedy strategy can be suboptimal because it may miss a better machine that was never adequately explored.

Exam Trap: Greedy = **pure exploitation** - only uses current knowledge, never explores. This can lead to suboptimal solutions!

What is the "random strategy" in the Multi-Armed Bandit problem?

- ① Always choosing the slot machine with the best payout history.
- ② Randomly selecting a slot machine to play each round (pure exploration).
- ③ Selecting actions based on the ϵ -greedy algorithm.
- ④ Choosing the slot machine with the highest estimated Q-value.

Answer: B Explanation: The random strategy explores by randomly selecting actions, ignoring past experience. Option A is the greedy strategy. Option C is ϵ -greedy. Option D is exploitation. While random exploration gathers information about all machines, it never exploits this knowledge to maximize reward.

Exam Trap: Random = **pure exploration** - never uses past knowledge. This wastes information and may perform poorly!

How does the ϵ -greedy strategy balance exploration and exploitation?

- ❶ It always explores with probability 1 and exploits with probability 0.
- ❷ With probability ϵ , it explores (random action); with probability $1 - \epsilon$, it exploits (best action so far).
- ❸ It alternates between exploration and exploitation in fixed rounds.
- ❹ It explores when the current reward is below a threshold.

Answer: B Explanation: ϵ -greedy explores with probability ϵ (random action) and exploits with probability $1 - \epsilon$ (best action).

Option A is pure exploration. Option C is not a standard approach. Option D is not how ϵ -greedy works. A common refinement is to start with a higher ϵ and decay it over time.

Exam Trap: ϵ = probability of exploration; $1 - \epsilon$ = probability of exploitation. Decaying ϵ over time is a common and effective strategy!

Q16: ϵ -Greedy Example

In the ϵ -greedy example, at Trial 2 with $\epsilon = 0.85$, a random number 0.99 is drawn. What decision is made and why?

- 1 Explore because $0.99 < 0.85$.
- 2 Exploit because $0.99 > 0.85$, so the agent chooses the machine with the highest Q-value.
- 3 Randomly choose between exploration and exploitation.
- 4 Choose a random machine because the random number is greater than 0.85.

Answer: B Explanation: Since $0.99 > 0.85$, the agent exploits (chooses the best action so far, which is Machine A). If the random number were less than ϵ , the agent would explore. The comparison is always: random number $< \epsilon \rightarrow$ explore; random number $\geq \epsilon \rightarrow$ exploit.

Exam Trap: If random number $< \epsilon$, **explore**; if random number $\geq \epsilon$, **exploit**. Don't reverse this logic!

In the MAB problem, after playing Machine A twice with rewards 1.2 and 0.8, what is the updated Q-value for Machine A?

- 1 $Q(A) = (1.2 + 0.8) / 2 = 1.0$
- 2 $Q(A) = 1.2 + 0.8 = 2.0$
- 3 $Q(A) = 1.2 - 0.8 = 0.4$
- 4 $Q(A) = \max(1.2, 0.8) = 1.2$

Answer: A Explanation: The Q-value is updated as the average of observed rewards: $Q(A) = (1.2 + 0.8) / 2 = 1.0$. Option B sums the rewards. Option C takes the difference. Option D takes the maximum. The general formula is: $Q_k^n = \frac{1}{n} \sum_{j=1}^n R_j$ for the n trials of action k .

Exam Trap: Q-values are **averages of observed rewards**, not sums or maximums!

What is the incremental update formula for Q-values in the MAB problem?

❶ $Q_k^n = Q_k^{n-1} + \frac{1}{n}(R_n - Q_k^{n-1})$

❷ $Q_k^n = Q_k^{n-1} + \alpha(R_n - Q_k^{n-1})$

❸ $Q_k^n = \frac{1}{n} \sum_{j=1}^n R_j$

❹ $Q_k^n = Q_k^{n-1} + R_n$

Answer: A Explanation: The incremental update formula is $Q_k^n = Q_k^{n-1} + \frac{1}{n}(R_n - Q_k^{n-1})$, where n is the number of times action k has been taken. This updates the Q-value by moving it a small step toward the new reward. Option B uses a constant α (used in Monte Carlo/TD methods). Option C is the non-incremental average. Option D is just adding the reward.

Exam Trap: For MAB, the update uses $\frac{1}{n}$ (decreasing step size). For MC/TD, it uses a constant α (learning rate). Don't confuse these!

In the MAB example with $\beta = 0.85$, what is the value of ϵ at trial t ?

- 1 $\epsilon_t = \beta^t$
- 2 $\epsilon_t = \beta^{t-1}$
- 3 $\epsilon_t = 1 - \beta^t$
- 4 $\epsilon_t = \beta \times t$

Answer: B Explanation: With exponential decay, $\epsilon_t = \beta^{t-1}$. For trial 1, $\epsilon_1 = \beta^0 = 1$ (pure exploration initially). For trial 2, $\epsilon_2 = \beta^1 = 0.85$. For trial 10, $\epsilon_{10} = 0.85^9 \approx 0.23$. This ensures more exploration early and more exploitation later.

Exam Trap: ϵ starts at 1 (trial 1) and decays; it does NOT start at β . The formula is $\epsilon_t = \beta^{t-1}$, not β^t !

Which of the following is a key challenge in reinforcement learning?

- ① RL algorithms require very little data to train effectively.
- ② RL algorithms are often data-hungry, requiring far more interaction with the environment than supervised learning algorithms.
- ③ RL algorithms never make errors during training.
- ④ RL algorithms do not require a reward function.

Answer: B Explanation: RL is notoriously data-hungry - often requiring millions of interactions. Option A is incorrect. Option C is false (RL makes many errors during exploration). Option D is false (a reward function is essential). The AlphaGo example required millions of self-play games.

Exam Trap: RL is **computationally expensive** and **data-hungry** - it needs many interactions to learn effectively!

A slot machine has been played 10 times with rewards: [0.8, 1.2, -0.5, 1.5, 0.9, -0.2, 1.1, 0.7, 1.3, 0.6]. What is the current Q-value for this machine?

- 1 0.74
- 2 0.80
- 3 0.90
- 4 1.00

Answer: A Explanation: Q-value = average of rewards = $(0.8 + 1.2 + (-0.5) + 1.5 + 0.9 + (-0.2) + 1.1 + 0.7 + 1.3 + 0.6) / 10 = 7.4 / 10 = 0.74$. The Q-value represents the expected reward from playing this machine. A higher Q-value indicates a machine with better expected payouts.

Exam Trap: Q-values are **averages** - include negative rewards in the calculation!

Three slot machines have the following Q-values: $Q(A) = 0.8$, $Q(B) = 0.5$, $Q(C) = 0.7$. If the agent uses the greedy strategy, which machine will it choose?

- 1 Machine A
- 2 Machine B
- 3 Machine C
- 4 A random machine

Answer: A Explanation: The greedy strategy chooses the action with the highest Q-value. $Q(A) = 0.8$ is the highest. Option B has the lowest Q-value. Option C is second best. Option D is the random strategy, not the greedy strategy.

Exam Trap: Greedy = highest Q-value only! If $Q(A) = 0.8$, $Q(B) = 0.5$, $Q(C) = 0.7$, always choose A. No exploration!

At trial 100 with $\epsilon = 0.10$, a random number 0.05 is drawn. Q-values are: $Q(A)=1.2$, $Q(B)=0.8$, $Q(C)=1.5$. What action does the agent take?

- ① Machine A (exploit)
- ② Machine B (exploit)
- ③ Machine C (exploit)
- ④ A random machine (explore)

Answer: D Explanation: Since $0.05 < 0.10$, the agent explores (chooses a random machine). Options A, B, C are all exploitation choices. The ϵ -greedy algorithm: if random number $< \epsilon \rightarrow$ explore; otherwise exploit with highest Q-value (Machine C if exploiting).

Exam Trap: Even if $Q(C)$ is highest, with $\epsilon = 0.10$ and random number 0.05, we explore! The random number determines the decision, not the Q-values.

Given $\beta = 0.95$ and exponential decay, what is ϵ at trial 50? (Round to 2 decimal places)

- 1 0.95
- 2 0.08
- 3 0.50
- 4 0.92

Answer: B Explanation: $\epsilon_t = \beta^{t-1} = 0.95^{49}$. $\ln(0.95^{49}) = 49 \times \ln(0.95) = 49 \times (-0.0513) = -2.513$. $e^{-2.513} \approx 0.081$.

So $\epsilon \approx 0.08$. This demonstrates how exploration decays over time.

Exam Trap: ϵ decays **exponentially** - use β^{t-1} , not $\beta \times t$!

Which strategy would be most likely to identify the true best slot machine in a MAB problem?

- ① Greedy strategy (pure exploitation)
- ② Random strategy (pure exploration)
- ③ ϵ -greedy strategy with decay
- ④ Always choosing the first machine

Answer: C **Explanation:** ϵ -greedy with decay balances exploration (to find the best machine) and exploitation (to maximize reward). Greedy may never find a better machine it never tries. Random never exploits knowledge. ϵ -greedy with decay is optimal: explores early when uncertainty is high, exploits later when knowledge is high.

Exam Trap: ϵ -greedy with decay is often the **best practical approach** - it explores when needed and exploits when confident!

In the MAB example, the true means were $\mu_A = 0.8$, $\mu_B = 0.5$, $\mu_C = 0.7$. After 10 trials, the estimated Q-values were $Q(A)=0.8$, $Q(B)=0.5$, $Q(C)=0.7$. Why might the estimated Q-values differ from the true means?

- ① They will always exactly equal the true means.
- ② Because the agent has played each machine a finite number of times, sampling error causes estimates to differ from true values.
- ③ Because the agent used the random strategy.
- ④ Because the true means are unknown and cannot be estimated.

Answer: B Explanation: Estimated Q-values are averages of observed samples. With a finite sample, they may differ from true means due to sampling error. As the number of samples increases, estimates converge to the true means (Law of Large Numbers). Option A is false. Option C is not the reason for estimation error. Option D is false (estimates are possible).

Exam Trap: Estimates are **averages of observed rewards**; they **may not equal** true means with small samples!

Why is exploration important in the Multi-Armed Bandit problem?

- ❶ Exploration guarantees immediate high rewards.
- ❷ Without exploration, the agent may miss a machine that could yield higher rewards than the currently estimated best.
- ❸ Exploration is always better than exploitation.
- ❹ Exploration is only needed in the first trial.

Answer: B Explanation: Exploration is critical to discover potentially better actions. The agent may have a high Q-value estimate for one machine due to lucky early rewards, while another machine has a higher true expected reward. Without exploring, the agent would never discover the better machine. Option A is false. Option C is false (balance is needed). Option D is false.

Exam Trap: Without exploration, you risk **suboptimal convergence** - never discovering the truly best action!

Why is exploitation important in the Multi-Armed Bandit problem?

- ❶ Exploitation is never important; only exploration matters.
- ❷ Exploitation uses knowledge gained from exploration to maximize immediate and long-term rewards.
- ❸ Exploitation guarantees discovering the best machine.
- ❹ Exploitation is only important when $\epsilon = 0$.

Answer: B Explanation: Exploitation leverages the knowledge gained from exploration to choose actions that are expected to yield high rewards. Without exploitation, the agent would never benefit from what it has learned. Option A is false. Option C is false (exploration discovers the best machine). Option D is false.

Exam Trap: Exploration and exploitation are **both necessary** - exploitation without exploration is greedy and may miss better options; exploration without exploitation wastes knowledge!

Which of the following is TRUE about the state in the Multi-Armed Bandit problem?

- ❶ The state changes after each action.
- ❷ There is only a single, constant state because the agent's actions do not affect the environment.
- ❸ The state is determined by the agent's policy.
- ❹ The state is the total reward accumulated so far.

Answer: B Explanation: In MAB, the environment is static - the slot machines' properties don't change regardless of which machine is played. Therefore, there is only one state. Option A describes more complex RL problems. Option C is incorrect (the policy maps states to actions). Option D is incorrect (state is not the accumulated reward).

Exam Trap: MAB is special because it has **only ONE state** - this is why it's a good introduction to RL!

In the MAB example, the payoffs from machine i are normally distributed with mean μ_i and standard deviation 1. What does this mean for the agent?

- ❶ The agent knows the exact payout for each machine before playing.
- ❷ The agent only receives deterministic payouts.
- ❸ The agent receives stochastic (random) payouts around the true mean, so it must learn the expected value through repeated trials.
- ❹ The agent can perfectly predict future payouts.

Answer: C Explanation: The normal distribution means payouts are random with some variance. The agent observes noisy rewards and must estimate the true mean through averaging multiple observations. Option A and D are false (the agent doesn't know the mean). Option B is false (payouts are stochastic).

Exam Trap: Rewards in RL are often **stochastic** - the agent must learn through experience, not through perfect knowledge!

Machine A has been played 5 times with average $Q(A)=1.2$. On the 6th play, the reward is 0.5. What is the new $Q(A)$ using the incremental update formula?

- ① $1.2 + (1/6)(0.5 - 1.2) = 1.083$
- ② $1.2 + 0.5 = 1.7$
- ③ $(1.2 \times 5 + 0.5) / 6 = 1.083$
- ④ $1.2 - (1/6)(0.5 - 1.2) = 1.317$

Answer: A and C Explanation: Both A and C give the same result. The incremental update formula is

$Q^n = Q^{n-1} + \frac{1}{n}(R_n - Q^{n-1}) = 1.2 + \frac{1}{6}(0.5 - 1.2) = 1.2 - 0.1167 = 1.0833$. The average formula is $(1.2 \times 5 + 0.5) / 6 = 1.0833$. Option B sums (incorrect). Option D uses subtraction with wrong sign.

Exam Trap: Both formulas give the same result - incremental update is just a computationally efficient way to update the average!

Q32: MAB - ϵ Decay Example

With $\beta = 0.90$, what is ϵ at trial 10?

- 1 0.90
- 2 0.39
- 3 0.43
- 4 0.81

Answer: C Explanation: $\epsilon_t = \beta^{t-1} = 0.90^9$. $0.90^2 = 0.81$, $0.90^4 = 0.656$, $0.90^8 = 0.430$, $0.90^9 = 0.387$. Wait, $0.90^9 = 0.387$. Let me recalculate: $0.9^1=0.9$, $0.9^2=0.81$, $0.9^3=0.729$, $0.9^4=0.656$, $0.9^5=0.590$, $0.9^6=0.531$, $0.9^7=0.478$, $0.9^8=0.430$, $0.9^9=0.387$. So $\epsilon_{10} \approx 0.39$. Option B (0.39) is correct for trial 10.

Exam Trap: For trial t , use exponent $t - 1$, not t !

What is the effect of a smaller decay factor β (e.g., 0.85 vs 0.99) in the ϵ -greedy strategy?

- ① A smaller β means ϵ decays faster, so the agent explores less over time.
- ② A smaller β means ϵ decays slower, so the agent explores more over time.
- ③ β has no effect on exploration.
- ④ A smaller β increases the learning rate.

Answer: A Explanation: A smaller β (e.g., 0.85 vs 0.99) means ϵ decays faster. For $\beta = 0.85$, after 10 trials $\epsilon \approx 0.23$; for $\beta = 0.99$, after 10 trials $\epsilon \approx 0.91$. So smaller β = less exploration later. Option B reverses this. Option C is false. Option D confuses β with α .

Exam Trap: β controls exploration decay rate - smaller β = faster decay = less exploration later!

Q34: MAB - Optimal Strategy with Known Values

If the agent knew the true means $\mu_A = 0.8$, $\mu_B = 0.5$, $\mu_C = 0.7$, what would be the optimal strategy?

- ① Always play Machine A.
- ② Always play Machine B.
- ③ Always play Machine C.
- ④ Use ϵ -greedy to explore.

Answer: A Explanation: If the agent knew the true means, there would be no need for exploration. The optimal strategy is to always choose the machine with the highest expected reward: Machine A (0.8). This is the maximum expected reward. Option B and C have lower means. Option D would be unnecessary.

Exam Trap: The challenge of RL is that the agent **does NOT know** the true values - it must learn them through experience!

As the number of plays for a given machine increases, what happens to the Q-value estimate?

- ① It diverges from the true mean.
- ② It converges to the true mean (by the Law of Large Numbers).
- ③ It oscillates randomly forever.
- ④ It becomes more and more negative.

Answer: B Explanation: By the Law of Large Numbers, the sample average converges to the expected value (true mean) as the number of samples increases. The Q-value is the sample average of observed rewards. Option A is the opposite. Option C is false (it converges). Option D is false.

Exam Trap: With enough plays, Q-values **converge** to the true expected rewards - this is why exploration is eventually less needed!

Q36: MAB - Bellman Equation Simplification

Why is the Bellman equation simpler in the MAB problem compared to general RL?

- ① Because there are no states in MAB.
- ② Because there is only one state and actions do not affect future states, so $Q(s, a)$ reduces to just the expected immediate reward.
- ③ Because MAB uses no discount factor.
- ④ Because MAB has deterministic rewards.

Answer: B Explanation: In MAB, there is only one state and actions do not change the state. The Bellman equation simplifies because the future value term ($\max Q$ of next state) is not needed - $Q(s,a)$ just equals the expected reward of action a . Option A is false (there is one state). Option C is false (MAB can use a discount factor but it's less relevant). Option D is false.

Exam Trap: MAB simplifies RL because **actions don't affect the state** - no need to consider future states!

Thompson Sampling is an alternative to ϵ -greedy for the MAB problem. How does it work conceptually?

- 1 It always chooses the machine with the highest Q-value.
- 2 It maintains a probability distribution over each machine's expected reward and samples from these distributions to choose actions, balancing exploration and exploitation probabilistically.
- 3 It chooses machines randomly without any learning.
- 4 It uses a fixed exploration schedule.

Answer: B Explanation: Thompson Sampling maintains a Bayesian belief (probability distribution) about each machine's reward.

It samples a possible expected reward from each distribution and chooses the machine with the highest sampled value. This naturally balances exploration (machines with high uncertainty may be sampled high) and exploitation (machines with high expected values are likely chosen). Option A is greedy. Option C is random. Option D is a fixed schedule.

Exam Trap: Thompson Sampling is a **probabilistic** approach that naturally balances exploration and exploitation without a fixed ϵ parameter!

What is "regret" in the context of the Multi-Armed Bandit problem?

- ❶ The negative reward received from playing a machine.
- ❷ The difference between the reward obtained by the agent's strategy and the reward that would have been obtained by always playing the optimal machine.
- ❸ The total number of times the agent explored.
- ❹ The variance of the rewards.

Answer: B Explanation: Regret measures the opportunity cost of not knowing which machine is best. It is the cumulative difference between the rewards obtained and the rewards that would have been obtained if the agent had always played the optimal machine. The goal is to minimize regret. Option A is a negative reward. Option C is exploration count. Option D is variance.

Exam Trap: Regret is **cumulative** - it grows with each suboptimal choice. The goal of RL is to minimize regret!

Which of the following is a real-world application of the Multi-Armed Bandit framework?

- ① A/B testing in online advertising to determine which ad creative yields the highest click-through rate.
- ② Training a robot to walk.
- ③ Playing a full game of chess.
- ④ Image recognition.

Answer: A **Explanation:** A/B testing is a classic MAB application: test different options (ads), learn which performs best, and allocate more traffic to the best option while still exploring others. Option B and C involve dynamic states (not MAB). Option D is supervised learning.

Exam Trap: MAB applies to **static** problems where actions don't affect the environment - like ad testing, not robotic control!

Which of the following is NOT a characteristic of the Multi-Armed Bandit problem?

- ① There is a single, constant state.
- ② The agent's actions affect the future state of the environment.
- ③ The agent chooses among multiple actions with unknown reward distributions.
- ④ The goal is to maximize cumulative reward.

Answer: B Explanation: In MAB, the agent's actions do NOT affect the future state (the slot machines don't change). Option B describes more complex RL problems with dynamic environments. Option A is true (single state). Option C is true (unknown reward distributions). Option D is true (maximize cumulative reward).

Exam Trap: The key difference between MAB and general RL is that in MAB, **actions do NOT change the state!**

In the Monte Carlo method, what is an "episode"?

- ① A single action taken by the agent.
- ② A single reward received by the agent.
- ③ A complete sequence of states, actions, and rewards from a starting point to a terminal state.
- ④ The total cumulative reward for a single action.

Answer: C Explanation: An episode is a complete trajectory from start to finish (terminal state). For example, a complete game of chess from opening to checkmate. Option A is a single step. Option B is a single reward. Option D is the return. MC methods learn from complete episodes.

Exam Trap: MC methods require **complete episodes** - they cannot learn until an episode ends!

When does the Monte Carlo method update its value estimates?

- ① After each action taken by the agent.
- ② After each reward received.
- ③ Only at the end of an episode, after the final outcome is known.
- ④ At random times during the episode.

Answer: C Explanation: MC methods wait until the end of the episode. Only then does the agent know the total accumulated reward (G_t) and can update Q-values for all state-action pairs encountered. This is a key difference from TD methods which update after each step.

Exam Trap: MC updates **only at episode end** - this means no learning happens within the episode!

What is a key advantage of the Monte Carlo method in reinforcement learning?

- ① It can learn after every step.
- ② It is unbiased because it learns directly from actual rewards.
- ③ It works for continuous, non-episodic tasks.
- ④ It requires no exploration.

Answer: B Explanation: MC methods are unbiased because they use actual observed rewards (not estimated values of future states). Option A describes TD methods. Option C is false (MC requires episodic tasks). Option D is false (MC still needs exploration).

Exam Trap: MC's advantage is **unbiased estimates**; its disadvantage is **high variance**!

What is a key disadvantage of the Monte Carlo method in reinforcement learning?

- ① It is biased because it uses bootstrapping.
- ② It suffers from high variance in estimates because returns can fluctuate significantly between episodes.
- ③ It can learn from incomplete episodes.
- ④ It requires no terminal state.

Answer: B Explanation: MC methods have high variance because each complete episode yields potentially very different total returns. Option A is false (MC is unbiased, not biased). Option C is false (MC requires complete episodes). Option D is false (MC requires a terminal state).

Exam Trap: MC = unbiased but high variance; TD = biased but low variance. This is the classic bias-variance trade-off in RL!

In the Monte Carlo method, how is $Q(s,a)$ updated?

- ① $Q(s, a) = Q(s, a) + \alpha(R_{t+1} + \gamma V(s') - Q(s, a))$
- ② $Q(s, a) = Q(s, a) + \alpha(G_t - Q(s, a))$
- ③ $Q(s, a) = \frac{1}{n} \sum R_j$
- ④ $Q(s, a) = \max(Q(s, a), G_t)$

Answer: B Explanation: MC updates $Q(s,a)$ towards the total observed return G_t from that point to the end of the episode:

$Q(s, a) \leftarrow Q(s, a) + \alpha(G_t - Q(s, a))$. Option A is the TD update. Option C is the MAB average. Option D takes the maximum.

Exam Trap: MC uses G_t (total episode return); TD uses $R_{t+1} + \gamma V(s')$ (one-step lookahead). This is the key difference!

When does the Temporal Difference (TD) method update its value estimates?

- ❶ Only at the end of an episode.
- ❷ After each step (or after a few steps), using the current estimate of the future value.
- ❸ Only when the agent reaches a terminal state.
- ❹ At random intervals.

Answer: B Explanation: TD methods update after each step (online learning). They use "bootstrapping" - updating the current value estimate using the estimate of the next state's value. Option A describes MC. Option C is episode-end. Option D is not standard.

Exam Trap: TD updates **every step** - it learns online, not just at episode end!

What is a key advantage of the Temporal Difference method over Monte Carlo?

- ① TD is unbiased.
- ② TD can learn from incomplete episodes and continuous (non-episodic) tasks.
- ③ TD has higher variance than MC.
- ④ TD requires less computation.

Answer: B Explanation: TD methods can learn online, from incomplete episodes (no need to wait for terminal state), and can handle continuous tasks. Option A is false (TD is biased). Option C is false (TD has lower variance). Option D is not necessarily true.

Exam Trap: TD is more **flexible** - can learn after every step and handle non-episodic problems!

What is a key disadvantage of the Temporal Difference method compared to Monte Carlo?

- ① TD has high variance.
- ② TD is biased because it uses bootstrapping (estimates of future values) rather than actual rewards.
- ③ TD cannot learn from incomplete episodes.
- ④ TD requires complete episodes.

Answer: B Explanation: TD methods are biased because they use bootstrapping - they update using estimates of future values (which are themselves estimates). MC is unbiased because it uses actual observed returns. Option A is false (TD has lower variance). Option C and D are false (TD can learn online).

Exam Trap: TD = **biased but lower variance**; MC = **unbiased but higher variance**!

What is the TD update formula for the value function $V(s)$?

- ① $V(s) \leftarrow V(s) + \alpha(G_t - V(s))$
- ② $V(s) \leftarrow V(s) + \alpha(R_{t+1} + \gamma V(s') - V(s))$
- ③ $V(s) \leftarrow \frac{1}{n} \sum R_j$
- ④ $V(s) \leftarrow R_{t+1} + \gamma V(s')$

Answer: B Explanation: TD updates $V(s)$ towards the TD target $R_{t+1} + \gamma V(s')$:

$V(s) \leftarrow V(s) + \alpha(R_{t+1} + \gamma V(s') - V(s))$. Option A is MC. Option C is MAB. Option D is just the target, not the update.

Exam Trap: TD uses **bootstrapping** - the target includes the estimate of the next state's value $V(s')$!

What is Q-learning?

- ❶ A Monte Carlo method that learns from complete episodes.
- ❷ A Temporal Difference method that learns the optimal action-value function $Q(s,a)$ directly, without waiting for episode end.
- ❸ A policy-based method that directly learns the policy.
- ❹ A model-based method that uses environment transition probabilities.

Answer: B Explanation: Q-learning is a TD method that learns $Q(s,a)$ using one-step lookahead:

$Q(s, a) \leftarrow Q(s, a) + \alpha(R_{t+1} + \gamma \max_{a'} Q(s', a') - Q(s, a))$. It is model-free and learns the optimal Q-function. Option A is MC. Option C is policy-based. Option D is model-based.

Exam Trap: Q-learning is **off-policy** - it learns the optimal policy regardless of the policy being followed!

What is the Q-learning update formula?

- ① $Q(s, a) \leftarrow Q(s, a) + \alpha(G_t - Q(s, a))$
- ② $Q(s, a) \leftarrow Q(s, a) + \alpha(R_{t+1} + \gamma Q(s', a') - Q(s, a))$
- ③ $Q(s, a) \leftarrow Q(s, a) + \alpha(R_{t+1} + \gamma \max_{a'} Q(s', a') - Q(s, a))$
- ④ $Q(s, a) \leftarrow R_{t+1} + \gamma \max_{a'} Q(s', a')$

Answer: C Explanation: Q-learning updates using $R_{t+1} + \gamma \max_{a'} Q(s', a')$ as the target. It uses the maximum Q-value over all actions in the next state (the optimal future value). Option A is MC. Option B is SARSA (uses the actual next action). Option D is just the target.

Exam Trap: Q-learning uses $\max_{a'} Q(s', a')$ (off-policy); SARSA uses $Q(s', a')$ for the actual action taken (on-policy). This is a key difference!

What is SARSA in reinforcement learning?

- ① A Monte Carlo method for policy evaluation.
- ② An on-policy Temporal Difference method that updates $Q(s,a)$ using the actual next action taken a' .
- ③ An off-policy TD method that uses the maximum next Q-value.
- ④ A model-based method for dynamic programming.

Answer: B Explanation: SARSA is an on-policy TD method. The update is

$Q(s, a) \leftarrow Q(s, a) + \alpha(R_{t+1} + \gamma Q(s', a') - Q(s, a))$, where a' is the action actually taken in state s' (following the current policy). Option A is MC. Option C is Q-learning. Option D is model-based.

Exam Trap: SARSA's name comes from the tuple: State, Action, Reward, State', Action'. It's on-policy!

What is the key difference between off-policy and on-policy reinforcement learning methods?

- ① Off-policy methods learn from experience generated by a different policy, while on-policy methods learn from experience generated by the policy being learned.
- ② On-policy methods are always better than off-policy methods.
- ③ Off-policy methods require complete episodes, while on-policy methods do not.
- ④ On-policy methods use value functions, while off-policy methods use policies directly.

Answer: A Explanation: In off-policy learning (e.g., Q-learning), the agent learns the optimal policy while following a different policy (e.g., ϵ -greedy). In on-policy learning (e.g., SARSA), the agent learns the policy it is currently following. Option B is false. Option C is false. Option D is false.

Exam Trap: Q-learning is **off-policy**; SARSA is **on-policy**. This is a key distinction for exam questions!

What is the TD error in the Temporal Difference method?

- ❶ The difference between the observed reward and the expected reward.
- ❷ The difference between the TD target and the current value estimate: $\delta = R_{t+1} + \gamma V(s') - V(s)$.
- ❸ The total return from an episode.
- ❹ The difference between the Q-values of two actions.

Answer: B Explanation: The TD error is the difference between the TD target and the current estimate:

$\delta = R_{t+1} + \gamma V(s') - V(s)$. It represents how much the new information (reward + next state estimate) differs from the current estimate. Option A is the prediction error. Option C is the return. Option D is the advantage.

Exam Trap: The TD error is the "surprise" - how much the new information differs from the current estimate!

In the NIM game example, after the first episode (Player 1 wins), how does the MC method update $Q(6,3)$ versus the TD method?

- ① MC updates $Q(6,3)$ using the total return $G = 1$; TD updates $Q(6,3)$ using the one-step lookahead $R_{t+1} + \gamma V(s')$.
- ② MC and TD update $Q(6,3)$ in exactly the same way.
- ③ MC does not update $Q(6,3)$ until the episode ends; TD also waits until episode ends.
- ④ MC updates $Q(6,3)$ after each step; TD waits until the episode ends.

Answer: A

Explanation: In the NIM example, MC updated $Q(6,3)$ using the total return $G = 1$ (from the entire episode). TD updated $Q(6,3)$ using the one-step lookahead $R_{t+1} + \gamma V(s')$, where s' is the next state after the action. Option B is false (they differ). Option C is false (TD updates online). Option D reverses the methods.

Exam Trap: MC uses G_t (total return); TD uses $R_{t+1} + \gamma V(s')$ (one-step lookahead).

After many episodes in the NIM game, both MC and TD methods converged to the same optimal strategy. What does this illustrate?

- ❶ MC and TD always converge to different strategies.
- ❷ MC and TD are both valid methods that converge to the optimal policy when enough experience is available.
- ❸ TD is always faster than MC.
- ❹ MC is always more accurate than TD.

Answer: B Explanation: The NIM example shows that both MC and TD converge to the same optimal strategy (pick 2 when 6 coins, pick 1 when 4 coins, pick 3 when 3 coins, etc.). This demonstrates that both methods are valid. Option A is false. Option C and D are not universally true.

Exam Trap: Both MC and TD can find the optimal policy - the choice depends on the problem characteristics!

In the MC update formula $Q(s, a) \leftarrow Q(s, a) + \alpha(G_t - Q(s, a))$, what does α represent?

- 1 The discount factor.
- 2 The learning rate, controlling how much the Q-value moves toward the new target.
- 3 The exploration rate.
- 4 The number of episodes.

Answer: B Explanation: α is the learning rate (step size), typically between 0 and 1. It controls how much the Q-value is updated toward the target. A larger α means faster learning but more sensitivity to noise. Option A is γ (discount factor). Option C is ϵ (exploration rate). Option D is n .

Exam Trap: α = learning rate; γ = discount factor; ϵ = exploration rate. Don't confuse these three!

In the TD update formula $V(s) \leftarrow V(s) + \alpha(R_{t+1} + \gamma V(s') - V(s))$, what is the TD target?

- ① $V(s)$
- ② $R_{t+1} + \gamma V(s')$
- ③ $\alpha(R_{t+1} + \gamma V(s') - V(s))$
- ④ R_{t+1}

Answer: B Explanation: The TD target is $R_{t+1} + \gamma V(s')$ - the immediate reward plus the discounted estimated value of the next state. This is what the algorithm tries to move $V(s)$ toward. Option A is the old value. Option C is the update amount. Option D is just the reward.

Exam Trap: The TD target is $R_{t+1} + \gamma V(s')$; the TD error is **target - current estimate**!

Which of the following tasks would be suitable for the Monte Carlo method?

- ❶ A robot continuously learning to navigate a warehouse without stopping.
- ❷ A game of chess that always has a defined ending (checkmate).
- ❸ A stock trading algorithm that runs indefinitely.
- ❹ A temperature control system that runs 24/7.

Answer: B **Explanation:** MC requires episodic tasks - those with a defined terminal state. Chess has a clear end. Options A, C, and D are continuous (non-episodic) tasks for which MC is not suitable. TD methods are better for continuous tasks.

Exam Trap: MC is **only for episodic** tasks; TD can handle both episodic and continuous tasks!

Which method is more suitable for a continuous, non-episodic task like a stock trading algorithm?

- ① Monte Carlo (MC) method.
- ② Temporal Difference (TD) method.
- ③ Both MC and TD are equally suitable.
- ④ Neither MC nor TD can handle continuous tasks.

Answer: B Explanation: TD methods are suitable for continuous tasks because they learn online, after each step, without needing to wait for an episode to end. MC requires complete episodes and cannot be applied to continuous tasks. Option C is false. Option D is false.

Exam Trap: TD is more **versatile** - it can handle both episodic and continuous tasks; MC is **limited to episodic** tasks!

What is Deep Reinforcement Learning (Deep RL)?

- ❶ RL that uses only tabular Q-tables.
- ❷ RL that combines reinforcement learning with deep neural networks to approximate value functions or policies.
- ❸ RL that uses decision trees instead of neural networks.
- ❹ RL that works only for board games.

Answer: B Explanation: Deep RL uses neural networks to approximate functions like $Q(s,a)$ or the policy. This is necessary when state spaces are too large for tabular methods. Option A describes traditional RL. Option C is not deep RL. Option D is false.

Exam Trap: Deep RL uses **neural networks** to handle large state spaces where Q-tables are impractical!

What is a Deep Q-Network (DQN)?

- ❶ A Monte Carlo method for policy evaluation.
- ❷ A TD method combined with neural networks to approximate $Q(s,a)$, with experience replay and target networks for stability.
- ❸ A policy-based method that directly learns the policy.
- ❹ A model-based method that uses environment transition probabilities.

Answer: B Explanation: DQN is a TD method (Q-learning) extended with neural networks. Key innovations include experience replay (breaking correlation between samples) and a separate target network (stabilizing learning). Option A is MC. Option C is policy-based. Option D is model-based.

Exam Trap: DQN = Q-learning + Neural Networks + Experience Replay + Target Network. Know all components!

What is "experience replay" in DQN and why is it important?

- ❶ Storing all experiences and replaying them in order to speed up training.
- ❷ Storing experiences (s, a, r, s') in a replay memory and sampling randomly for training, which breaks the correlation between consecutive experiences.
- ❸ Replaying the game from the beginning after each episode.
- ❹ Storing only the last experience for training.

Answer: B Explanation: Experience replay stores past experiences in memory and samples randomly for training. This breaks the temporal correlation between consecutive experiences (which are highly similar in games), making learning more stable and efficient. Option A is incorrect (random sampling, not sequential). Option C is not experience replay. Option D is not experience replay.

Exam Trap: Experience replay = **random sampling** from memory, not sequential replay!

What is the purpose of the target network in DQN?

- ❶ To provide a stable target for Q-learning updates, preventing the "moving target" problem.
- ❷ To reduce the size of the neural network.
- ❸ To increase the learning rate.
- ❹ To replace the policy network entirely.

Answer: A Explanation: The target network is a copy of the policy network that is updated periodically (e.g., every 500 steps). It provides a stable target for the loss function, preventing the "chasing a moving target" problem that can cause instability. Option B is false. Option C is false. Option D is false.

Exam Trap: The target network is updated **periodically**, not every step - this stabilizes learning!

What is the typical loss function used in DQN?

- 1 Cross-entropy loss.
- 2 Mean Squared Error (MSE) between the policy network's Q-value and the target network's Q-value.
- 3 Absolute error loss.
- 4 Hinge loss.

Answer: B Explanation: DQN uses MSE loss: $L = (Q_{\text{policy}}(s, a) - (R_{t+1} + \gamma \max_{a'} Q_{\text{target}}(s', a')))^2$. This minimizes the squared difference between the policy network's prediction and the target. Option A is for classification. Option C is L1 loss. Option D is for SVM.

Exam Trap: DQN uses **MSE** between policy and target networks, not cross-entropy!

How does DQN typically balance exploration and exploitation during training?

- ❶ It uses pure exploitation only.
- ❷ It uses pure exploration only.
- ❸ It uses ϵ -greedy with ϵ typically decaying over time.
- ❹ It always chooses actions randomly.

Answer: C Explanation: DQN typically uses ϵ -greedy exploration with ϵ decaying from a high value (e.g., 1.0) to a low value (e.g., 0.01) over training. This encourages exploration early and exploitation later. Option A and B are extremes. Option D is the random strategy.

Exam Trap: DQN uses ϵ -greedy with decay - same principle as MAB but applied in complex environments!

When does DQN typically stop training?

- ❶ After exactly 100 steps.
- ❷ When the maximum number of steps is reached or there is no significant improvement in Q-values.
- ❸ When the agent wins the game.
- ❹ When the loss function reaches zero.

Answer: B Explanation: DQN training typically ends when the maximum number of steps is reached or when Q-values stop improving (convergence). Option A is arbitrary. Option C may not correspond to convergence. Option D is unrealistic (loss rarely reaches zero).

Exam Trap: DQN training stops when **converged** or **max steps reached** - not necessarily when the agent wins!

What is the key difference between model-based and model-free reinforcement learning?

- ❶ Model-based uses a model of the environment's dynamics; model-free learns directly from experience without an environment model.
- ❷ Model-based is always better than model-free.
- ❸ Model-free uses a model of the environment; model-based does not.
- ❹ Model-based is only for games; model-free is for robotics.

Answer: A Explanation: Model-based RL uses a model of the environment (transition probabilities, rewards) to plan actions.

Model-free RL learns directly from experience without a model. Option B is false. Option C reverses them. Option D is false.

Exam Trap: Model-based = [uses environment model](#); Model-free = [learns from experience](#)!

What is a key advantage of model-based reinforcement learning?

- ❶ It requires less computational power.
- ❷ It can be more efficient because the agent requires fewer interactions with the environment.
- ❸ It always outperforms model-free RL.
- ❹ It requires no reward function.

Answer: B Explanation: Model-based RL can be more sample-efficient because the agent can use the model to simulate and plan, requiring fewer actual interactions with the environment. Option A is false (model-based can be computationally expensive). Option C is false. Option D is false.

Exam Trap: Model-based RL is **more sample-efficient** but can suffer from model bias!

What is a key disadvantage of model-based reinforcement learning?

- ❶ It requires no environment knowledge.
- ❷ Modeling real-world environments can be extremely difficult, leading to inaccuracies and biases that can be exploited by the agent.
- ❸ It always learns faster than model-free RL.
- ❹ It cannot use neural networks.

Answer: B Explanation: The main challenge is that modeling complex environments is difficult and inaccurate. Model bias can cause the agent to learn a policy that works well in the model but poorly in the real world. Option A is false. Option C is false. Option D is false (model-based can use neural networks).

Exam Trap: Model-based RL's risk is **model bias** - the model may not accurately represent the real environment!

What is a key advantage of model-free reinforcement learning?

- ① It requires fewer training samples than model-based RL.
- ② It learns directly from experience without needing an environment model, making it suitable for dynamic and unpredictable scenarios.
- ③ It always converges faster than model-based RL.
- ④ It requires less computational power.

Answer: B Explanation: Model-free RL learns directly from interaction with the environment. It doesn't require building a model, making it applicable to complex, dynamic, or poorly understood environments. Option A is false (model-free is typically less sample-efficient). Option C is false. Option D is not necessarily true.

Exam Trap: Model-free RL is **more applicable** to complex environments but **less sample-efficient!**

What is a key disadvantage of model-free reinforcement learning?

- ① It cannot be used for dynamic environments.
- ② It tends to have a slow learning process because it requires many interactions with the environment.
- ③ It requires an accurate environment model.
- ④ It is only suitable for deterministic environments.

Answer: B Explanation: Model-free RL is data-hungry, requiring many interactions with the environment to learn. It can be very slow, especially for complex problems. Option A is false (it works in dynamic environments). Option C describes model-based RL. Option D is false.

Exam Trap: Model-free RL's main drawback is **slow learning** due to many required interactions!

AlphaZero is an example of which type of reinforcement learning?

- ① Model-free RL
- ② Model-based RL
- ③ Supervised learning
- ④ Unsupervised learning

Answer: B Explanation: AlphaZero uses model-based RL. It has a model of the game (rules, transitions) and uses Monte Carlo Tree Search (MCTS) to plan and evaluate moves. It demonstrates superhuman performance in chess, Go, and shogi. Option A is incorrect. Options C and D are not RL.

Exam Trap: AlphaZero is **model-based** RL - it uses the game's rules as its model!

What makes AlphaGo a significant achievement in reinforcement learning?

- ❶ It was the first computer program to play Go.
- ❷ It learned to play at a level surpassing the best human players purely by playing against itself millions of times.
- ❸ It used only supervised learning.
- ❹ It required no computational resources.

Answer: B Explanation: AlphaGo's significance is that it achieved superhuman performance through self-play (reinforcement learning), without relying on human gameplay data for its final strategy. Option A is false. Option C is false (it used RL primarily). Option D is false.

Exam Trap: AlphaGo demonstrated that RL can **discover novel strategies** beyond human knowledge!

What is Reinforcement Learning from Human Feedback (RLHF)?

- ❶ RL that uses only automated rewards from the environment.
- ❷ RL that uses human feedback (rankings or scores) as a reward signal to fine-tune models, particularly large language models like ChatGPT.
- ❸ RL that uses only supervised learning.
- ❹ RL that requires no training data.

Answer: B Explanation: RLHF uses human feedback as the reward signal. In the context of LLMs, humans rank or score model outputs, and RL algorithms adjust the model to generate responses that receive higher scores. This improves instruction following and coherence. Option A is standard RL. Option C is not RL. Option D is false.

Exam Trap: RLHF is a key technique for [fine-tuning LLMs](#) - it's why ChatGPT and similar models are so capable!

What is the primary purpose of RLHF in large language models (LLMs)?

- ❶ To reduce the size of the model.
- ❷ To encourage the model to follow instructions, respect policy constraints, and reason more coherently.
- ❸ To increase the model's training speed.
- ❹ To replace all other training methods.

Answer: B Explanation: RLHF fine-tunes LLMs to better align with human preferences, improving instruction following, reasoning, and safety. Option A is not a purpose. Option C is false. Option D is false (RLHF is a fine-tuning step, not a replacement).

Exam Trap: RLHF is about **alignment** - making AI systems behave in ways humans prefer!

What is the key difference between value-based and policy-based reinforcement learning methods?

- ① Value-based methods learn the optimal policy directly; policy-based methods learn the value function.
- ② Value-based methods learn a value function (V or Q) and derive a policy from it; policy-based methods learn the policy directly without learning a value function.
- ③ Value-based methods are always better than policy-based methods.
- ④ Policy-based methods cannot handle continuous action spaces.

Answer: B Explanation: Value-based methods (Q-learning, SARSA) learn value functions and derive policies from them (e.g., choose action with highest Q). Policy-based methods learn the policy directly, often parameterized by neural networks. Option A reverses them. Option C is false. Option D is false.

Exam Trap: Policy-based = **directly learns the policy**; Value-based = **learns values then derives policy**!

What is a key advantage of policy-based methods over value-based methods?

- ① They are simpler to implement.
- ② They can handle continuous action spaces more naturally.
- ③ They always converge faster.
- ④ They require no exploration.

Answer: B Explanation: Policy-based methods can handle continuous action spaces naturally because they output a probability distribution over actions (or a deterministic action), which works for continuous spaces. Option A is not universally true. Option C is false. Option D is false.

Exam Trap: Policy-based methods are preferred for **continuous action spaces** (e.g., robotic control)!

What is a key disadvantage of policy-based methods?

- ① They cannot handle continuous action spaces.
- ② They may be slow to converge and are sensitive to hyperparameters like the learning rate.
- ③ They always require complete episodes.
- ④ They cannot use neural networks.

Answer: B Explanation: Policy-based methods often converge slowly and are sensitive to hyperparameters. Some variants (like REINFORCE) also require complete episodes. Option A is false (they handle continuous spaces well). Option C is only true for some variants. Option D is false.

Exam Trap: Policy-based methods can be **slow to converge** and **sensitive to tuning**!

What are Actor-Critic methods in reinforcement learning?

- ① Value-based methods that only learn value functions.
- ② Policy-based methods that only learn policies.
- ③ Hybrid approaches that combine value-based and policy-based methods, using an "actor" (policy) and a "critic" (value function).
- ④ Model-based methods that use environment models.

Answer: C Explanation: Actor-Critic methods combine the strengths of both approaches: the "actor" learns the policy (policy-based), while the "critic" evaluates the value function (value-based) to provide feedback to the actor. This hybrid approach can be more stable and efficient. Option A is value-based only. Option B is policy-based only. Option D is model-based.

Exam Trap: Actor-Critic = Actor (policy) + Critic (value) - the best of both worlds!

What is a Markov Decision Process (MDP)?

- ❶ A process where future states depend on all past states.
- ❷ A mathematical framework for modeling decision-making where future states depend only on the current state (Markov property).
- ❸ A process with no states.
- ❹ A process with no actions.

Answer: B Explanation: MDPs have the Markov property: the future state depends only on the current state, not on the history of past states. This "memoryless" property simplifies the analysis and makes problems tractable. Option A describes non-Markovian processes. Option C is false. Option D is false.

Exam Trap: MDPs assume the **Markov property** - only the current state matters!

What does the Markov property state in reinforcement learning?

- ❶ $P(S_{t+1}|S_t, S_{t-1}, \dots, S_1) = P(S_{t+1}|S_t)$
- ❷ $P(S_{t+1}|S_t, S_{t-1}, \dots, S_1) = P(S_{t+1}|S_t, S_{t-1})$
- ❸ $P(S_{t+1}|S_t) = 1$
- ❹ $P(S_{t+1}|S_t) = 0$

Answer: A Explanation: The Markov property states that the probability of transitioning to the next state depends only on the current state, not on the history of past states. This is the "memoryless" property. Options B, C, and D are incorrect.

Exam Trap: Markov property = **only current state matters** for predicting the future!

What is the Bellman equation in reinforcement learning?

- ① An equation for computing the loss function in supervised learning.
- ② A recursive equation that expresses the value of a state in terms of immediate reward and the discounted value of the next state.
- ③ An equation for calculating the learning rate.
- ④ An equation for determining the exploration rate.

Answer: B Explanation: The Bellman equation provides a recursive way to evaluate policies: $V(s) = R(s, a) + \gamma V(s')$ for deterministic environments. It decomposes the total return into immediate reward plus future returns. Option A is for supervised learning. Option C is not Bellman. Option D is not Bellman.

Exam Trap: The Bellman equation is **recursive** - $V(s)$ depends on $V(s')$!

What is the Bellman optimality equation for the state-value function $V(s)$?

- ① $V^*(s) = \max_a E[R_{t+1} + \gamma V^*(s')]$
- ② $V^*(s) = E[R_{t+1} + \gamma V^*(s')]$
- ③ $V^*(s) = \max_a R_{t+1}$
- ④ $V^*(s) = \max_a Q(s, a)$

Answer: A Explanation: The Bellman optimality equation for $V(s)$ is $V^*(s) = \max_a E[R_{t+1} + \gamma V^*(s')]$. It states that the optimal value of a state is the maximum over actions of the expected reward plus the discounted optimal value of the next state. Option B lacks the max. Option C is only immediate reward. Option D is the relationship between V and Q .

Exam Trap: The Bellman optimality equation includes **max over actions** - it assumes the agent chooses optimally!

What is the Bellman optimality equation for $Q(s,a)$?

❶ $Q^*(s, a) = E[R_{t+1} + \gamma \max_{a'} Q^*(s', a')]$

❷ $Q^*(s, a) = R_{t+1} + \gamma Q^*(s', a')$

❸ $Q^*(s, a) = \max_a E[R_{t+1} + \gamma V^*(s')]$

❹ $Q^*(s, a) = E[R_{t+1} + \gamma Q^*(s', a')]$

Answer: A Explanation: The Bellman optimality equation for $Q(s,a)$ is $Q^*(s, a) = E[R_{t+1} + \gamma \max_{a'} Q^*(s', a')]$. This is the fundamental equation behind Q-learning. Option B lacks the expectation and max. Option C is $V(s)$. Option D lacks the max.

Exam Trap: The Bellman optimality equation for Q uses $\max_{a'} Q^*(s', a')$ - the optimal future value!

In an MDP, what does the transition probability matrix P represent?

- ① The probability of each action being taken.
- ② The probability of moving from any state to any other state in one step.
- ③ The probability of receiving a specific reward.
- ④ The probability distribution over initial states.

Answer: B Explanation: The transition matrix P contains the probabilities of transitioning from one state to another. Given the Markov property, $P_{ij} = P(S_{t+1} = j | S_t = i)$. Option A is the policy. Option C is the reward distribution. Option D is the initial state distribution.

Exam Trap: P describes **state transitions** - the probability of moving between states!

In the NIM game example with 6 coins, how are states represented?

- ❶ By the number of coins left on the table.
- ❷ By the total number of coins removed.
- ❸ By the player's identity.
- ❹ By the action taken.

Answer: A **Explanation:** In the NIM game, the state is represented by the number of coins left on the table (1, 2, 3, 4, 5, 6).

Option B is the cumulative total. Option C is player identity (but the state is the same for both players). Option D is the action.

Exam Trap: The NIM game state is simply the **number of coins remaining** - simple and discrete!

In the NIM game example, what are the possible actions at each state?

- ❶ Remove any number of coins.
- ❷ Remove 1, 2, or 3 coins.
- ❸ Remove exactly 1 coin.
- ❹ Remove coins based on the opponent's move.

Answer: B **Explanation:** In the NIM game, players can remove 1, 2, or 3 coins each turn (as long as enough coins remain).

Option A is incorrect (not any number). Option C is too restrictive. Option D is not an action choice.

Exam Trap: Actions in NIM are 1, 2, or 3 coins - the agent chooses which number to remove!

After 1,000,000 episodes in the NIM game, the MC method converged to $Q(6,2) = 1$. What does this indicate?

- ❶ Picking 2 coins when 6 coins are left leads to certain loss.
- ❷ Picking 2 coins when 6 coins are left leads to certain win.
- ❸ Picking 2 coins when 6 coins are left is not a legal action.
- ❹ The action value is uncertain.

Answer: B Explanation: $Q(6,2) = 1$ indicates that taking 2 coins from 6 coins leads to a win (reward of 1). This is the optimal strategy identified by the algorithm. The Q-value represents the expected future reward from that state-action pair.

Exam Trap: Q-values near 1 = winning actions; Q-values near -1 = losing actions!

What is the optimal strategy learned for the NIM game with 6 coins?

- ❶ Pick 3 coins when 6 coins remain.
- ❷ Pick 2 coins when 6 coins remain.
- ❸ Pick 1 coin when 6 coins remain.
- ❹ Any action is optimal.

Answer: B Explanation: The optimal strategy learned is to pick 2 coins when 6 coins remain (leaving 4 coins), then respond optimally (pick 1 coin when 4 remain, pick 3 when 3 remain, etc.). This strategy ensures a win. Option A is suboptimal. Option C is suboptimal. Option D is false.

Exam Trap: The NIM game strategy is: 6 coins $\rightarrow$ pick 2; 4 coins $\rightarrow$ pick 1; 3 coins $\rightarrow$ pick 3; 2 coins $\rightarrow$ pick 2; 1 coin $\rightarrow$ pick 1!

In the NIM game example, after the first episode, why did MC update $Q(1,1)$ but TD also updated $Q(1,1)$ at the same time?

- 1 MC updated at episode end; TD updated within the episode (after the winning move).
- 2 MC and TD both wait until episode end.
- 3 MC updates within episodes; TD waits until episode end.
- 4 Neither method updated $Q(1,1)$.

Answer: A Explanation: In the NIM example, MC updated $Q(1,1)$ at the end of the episode using $G=1$. TD updated $Q(1,1)$ within the episode (after the winning move was taken) using the reward $R=1$. This illustrates the timing difference: MC $\rightarrow$ episode end; TD $\rightarrow$ within episode.

Exam Trap: MC updates at **episode end**; TD updates **within the episode** (after each step)!

In the NIM MC example, $Q(6,3)$ was updated from 0 to 0.05 after the first episode. What does this represent?

- ❶ $Q(6,3)$ now equals the true value.
- ❷ $Q(6,3)$ was moved slightly toward the observed return $G=1$ using $\alpha = 0.05$.
- ❸ $Q(6,3)$ was set to 1.0.
- ❹ $Q(6,3)$ was not updated.

Answer: B Explanation: The update was $Q(6, 3) = 0 + 0.05(1 - 0) = 0.05$. This moves the Q-value toward the observed return ($G=1$) by a small amount. Option A is false (true value takes many episodes). Option C is false. Option D is false.

Exam Trap: MC updates Q-values **gradually** toward observed returns, not setting them directly!

In the NIM TD example, $Q(6,3)$ was updated from 0 to 0.0025 after the third episode. Why is this value different from the MC update?

- ① TD uses the one-step lookahead $R_{t+1} + \gamma V(s')$ rather than the full episode return G .
- ② TD uses a different α parameter.
- ③ TD does not update $Q(6,3)$ at all.
- ④ TD uses the full episode return G .

Answer: A Explanation: TD updated $Q(6,3)$ using $R_{t+1} + \gamma V(1)$ where $V(1) = 0.05$ (from the previous episode). The TD target was $0 + 0.05 = 0.05$, so $Q(6,3) = 0 + 0.05(0.05 - 0) = 0.0025$. MC used $G=1$, giving 0.05. This difference is due to MC using the full episode return.

Exam Trap: MC uses G (full episode return); TD uses **one-step lookahead** - this creates different update values!

After 1,000,000 episodes, the NIM Q-table showed $Q(4,1) = 0.383$. What does this mean?

- ❶ Taking 1 coin from 4 coins is a bad move.
- ❷ Taking 1 coin from 4 coins leads to a good outcome (positive expected reward).
- ❸ Taking 1 coin from 4 coins is illegal.
- ❹ The Q-value is undefined.

Answer: B Explanation: $Q(4,1) = 0.383$ means that taking 1 coin from 4 coins has a positive expected reward (likely a winning move). The optimal strategy for 4 coins is to pick 1 coin. Option A is incorrect. Option C is false. Option D is false.

Exam Trap: Positive Q-values = good actions; Negative Q-values = bad actions!

Why are Q-values for taking 2 or 3 coins when only 1 coin remains zero in the NIM Q-table?

- ① Because those actions are illegal (you cannot take more coins than are available).
- ② Because those actions always win.
- ③ Because those actions always lose.
- ④ Because those actions have not been tried.

Answer: A Explanation: In the NIM game, you cannot take more coins than are available. So when 1 coin remains, taking 2 or 3 coins is illegal. The Q-values for illegal actions remain 0. Option B is false. Option C is false (they're illegal, not losing). Option D is not the reason.

Exam Trap: Illegal actions have Q-values of 0 **by convention** - they are never selected!

Which of the following is a significant challenge in applying RL to real-world problems?

- ① RL requires very little computational power.
- ② RL may require millions of interactions with the environment, which can be expensive or dangerous in real-world settings.
- ③ RL never makes mistakes during learning.
- ④ RL does not require a reward function.

Answer: B Explanation: A key challenge is the need for many interactions - millions of trials. In real-world settings (robotics, healthcare), this can be expensive, dangerous, or impossible. Option A is false (RL is computationally expensive). Option C is false. Option D is false.

Exam Trap: RL's data hunger is a major barrier to real-world deployment - [sample efficiency](#) is critical!

Why is reward design crucial in reinforcement learning?

- ❶ The reward function determines what the agent learns to optimize.
- ❷ The reward function has no effect on learning.
- ❸ Any reward function works equally well.
- ❹ Rewards are only used for exploration.

Answer: A Explanation: The reward function defines the objective - what the agent tries to maximize. Poorly designed rewards (reward hacking, sparse rewards) can lead to unintended behaviors. Option B is false. Option C is false. Option D is false.

Exam Trap: Reward design is critical - poorly designed rewards can cause unintended and harmful agent behavior!

What is the challenge of sparse rewards in reinforcement learning?

- ❶ Sparse rewards provide frequent feedback, making learning faster.
- ❷ Sparse rewards provide little feedback, making it difficult for the agent to learn because it may take many actions before receiving any reward.
- ❸ Sparse rewards always lead to better policies.
- ❹ Sparse rewards are not used in RL.

Answer: B Explanation: Sparse rewards (e.g., only reward at the end of a game) provide little signal for learning. The agent may need many episodes to discover which actions led to the reward. Dense rewards (feedback after each step) provide more information and accelerate learning. Option A is false. Option C is false. Option D is false.

Exam Trap: Sparse rewards make learning **difficult**; dense rewards make learning **easier**!

What is "sample efficiency" in reinforcement learning?

- ① The number of samples the agent needs to achieve good performance.
- ② The quality of samples used for training.
- ③ The speed of the training algorithm.
- ④ The size of the neural network.

Answer: A Explanation: Sample efficiency measures how many interactions with the environment are needed to learn a good policy. Model-based methods are often more sample-efficient. Off-policy methods can be more sample-efficient than on-policy methods. Option B is sample quality. Option C is training speed. Option D is network size.

Exam Trap: Sample efficiency is **how many experiences** the agent needs to learn well - a key practical concern!

Which of the following statements provides the most comprehensive summary of reinforcement learning?

- ❶ RL is a supervised learning method that uses labeled data to make predictions.
- ❷ RL is an unsupervised learning method that finds clusters in data.
- ❸ RL is a learning paradigm where an agent learns through trial and error to maximize cumulative reward, using concepts like policies, value functions, exploration vs exploitation, and methods like Monte Carlo and Temporal Difference. It differs from supervised and unsupervised learning in that it learns sequential decision-making strategies rather than static mappings.
- ❹ RL is a method that always outperforms supervised learning in all tasks.

Answer: C Explanation: Option C provides a comprehensive summary: agent learns from interaction (trial and error) to maximize cumulative reward; key concepts include policies, value functions, exploration vs exploitation; methods include MC and TD; it differs from supervised/unsupervised in that it learns decision strategies. Option A is false. Option B is false. Option D is false.

Exam Trap: RL is **distinct** from supervised and unsupervised learning - it's about **sequential decision-making** to maximize reward!

Top 10 RL Exam Traps to Remember:

- 1 **Trap 1:** RL output is a **policy** (strategy), not a prediction or classification
- 2 **Trap 2:** MAB has **ONLY ONE state** - actions don't change the environment
- 3 **Trap 3:** ϵ -greedy: random number $< \epsilon \rightarrow$ **explore**; $\geq \rightarrow$ **exploit**
- 4 **Trap 4:** ϵ decays as $\epsilon_t = \beta^{t-1}$ (not β^t)
- 5 **Trap 5:** MC updates **at episode end** using G_t ; TD updates **every step** using $R_{t+1} + \gamma V(s')$
- 6 **Trap 6:** MC = **unbiased but high variance**; TD = **biased but low variance**
- 7 **Trap 7:** Q-learning is **off-policy**; SARSA is **on-policy**
- 8 **Trap 8:** α = **learning rate**; γ = **discount factor**; ϵ = **exploration rate**
- 9 **Trap 9:** DQN uses **experience replay** + **target network** for stability
- 10 **Trap 10:** Policy-based methods handle **continuous action spaces**; value-based methods handle discrete actions

Remember: RL is about **learning optimal behavior through experience!** The exam tests conceptual understanding of the trade-offs between methods.